EC200U&EG91xU Series QuecOpen File System API Reference Manual

LTE Standard Module Series

Version: 1.1

Date: 2023-08-28

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

About the Document

Revision History

Version	Date	Author	Description
-	2021-10-08	Kevin WANG/ Herry GENG	Creation of the document
1.0	2021-10-21	Kevin WANG/ Herry GENG	First official released
1.1	2023-08-28	Kevin WANG/ Herry GENG	1. Added applicable modules EG912U-GL and EG915U series. 2. Updated the file system partitions (Chapters 2.2 & 3.3.1).

Contents

Table Index

1 Introduction

Quectel EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see ***document [1]***.

This document introduces the file system, file system API and relevant demo of EC200U and EG91xU family modules in QuecOpen® solution.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 Brief Introduction to File System

2.1. Design Principle

The file system API of EC200U series and EG91xU family modules is designed as close to the C standard library as possible. The internal file system of the module adopts SFFS format, and is therefore not compatible with any file system on PC or Linux.

2.2. File System Partitions

The file system of the module currently provides three disk partitions, namely UFS, EXNSFFS, EXNAND EFS, and SD.

- UFS: Located in the built-in flash, it is used to store common files. The SFS encrypted file directory for storing encrypted files is also in this partition.
- EXNSFFS: External 4-wire SPI NOR Flash partition, which requires external 4-wire SPI NOR Flash before being used.
- EXNAND: External SPI NAND FLASH partition, which requires external SPI NAND FLASH before being used.
- EFS: External 6-wire SPI NOR Flash partition, which requires external 6-wire SPI NOR Flash before being used.
- SD: SD card partition.

2.3. Functions Briefs

Below are the operations that can be performed in the file system of the module.

- File operations: Open, read, write, close, erase and rename files, move the file pointer, get the current position of the file pointer, move the file pointer back to the beginning, change and get file size, etc.
- Directory operations: Create, open, read, and close directories, erase empty directories, move the directory pointer, get the current position of the directory pointer, move the directory pointer back to the beginning, etc.
- Other operations: Get file status and the remaining space size of a partition, write and read configuration files, configure SFS file keys, write and read a character, synchronize modified data to a storage device, etc.

3 File System API

3.1. Header File

ql_fs.h, the header file of file system API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all the header files mentioned in this document are in this directory.

3.2. API Overview

Table 2: API Overview

Function	Description
ql_fopen()	Opens a file and returns the file handle.
ql_fclose()	Closes the specified file.
ql_remove()	Erases the specified file.
ql_fread()	Reads data from the specified file.
ql_fwrite()	Writes data to the specified file.
ql_fseek()	Moves the file pointer to a specific location in a file.
ql_frewind()	Moves the file pointer to the beginning of a file.
ql_ftell()	Returns relative byte offsets of the file pointer from the beginning of a file.
ql_fstat()	Gets file status with the file handle.
ql_stat()	Gets file status with the file name.
ql_ftruncate()	Truncates a file from the beginning of the file to the specified length.
ql_fsize()	Gets the file size.

ql_file_exist()	Checks whether a file exists.
ql_mkdir()	Creates a directory.
ql_opendir()	Opens a specified directory and returns the directory handle.
ql_closedir()	Closes a specified directory.
ql_readdir()	Gets file information from the specified directory.
ql_telldir()	Returns the position of the specified directory stream.
ql_seekdir()	Sets the directory stream read position.
ql_rewinddir()	Moves the directory stream read position to the beginning of the directory.
ql_rmdir()	Erases the specified empty directory.
ql_rename()	Renames a file.
ql_fs_free_size()	Gets the size of the remaining space in the indicated partition with a file (with path), directory path, or partition name.
ql_nvm_fwrite()	Writes a simple configuration file.
ql_nvm_fread()	Reads a simple configuration file.
ql_sfs_setkey()	Configures the SFS file key for reading and writing.
ql_fs_free_size_by_fd()	Gets the remaining space of the indicated partition with the file handle.
ql_fputc()	Writes a character to a file.
ql_fgetc()	Reads a character from a file.
ql_ferror()	Checks whether the current file handle is valid.
ql_fsync()	Synchronizes the modified file data in memory to a storage device.
ql_cust_nvm_fwrite()	Writes a user-defined simple configuration file.
ql_cust_nvm_fread()	Reads a user-defined simple configuration file.

3.3. API Description

3.3.1. ql_fopen

This function opens a file and returns the file handle. The file handle is a 32-bit signed integer.

- **Prototype**

```
QFILE ql_fopen(const char * file_name, const char *mode)
```

- **Parameter**

file_name:
[In] The file name (with path), which should not exceed 132 bytes. The format is "disk name:/directory path/file name". The following partitions are currently supported:

UFS:	Main disk, used to store user's common files.
SFS:	Encrypted disk, used to store user's encrypted files, shares a partition with UFS.
EFS:	External 6-wire SPI NOR Flash partition.
SD:	SD card partition.
EXNSFFS:	External 4-wire SPI NOR Flash partition.
EXNAND:	External SPI NAND FLASH partition.

mode:
[In] The file open mode, which is consistent with the C standard library. For example, "wb+".

- **Return Value**

File handle	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.2. ql_fclose

This function closes the specified file.

- **Prototype**

```
int ql_fclose(QFILE fd)
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.3. ql_remove

This function erases the specified file.

- **Prototype**

```
int ql_remove(const char *file_name)
```

- **Parameter**

file_name:
[In] The file name, whose format is consistent with the one in *ql_fopen()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.4. ql_fread

This function reads data from the specified file.

- **Prototype**

```
int ql_fread(void * buffer, size_t size, size_t num, QFILE fd)
```

- **Parameter**

buffer:
[In] The memory address for storing the data to be read, which cannot be a null pointer.

size:
[In] The size of each element to be read. Unit: byte.

num:
[In] The number of the elements to be read. The final size of data read is *size* × *num*.

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

The size of data read	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.5. ql_fwrite

This function writes data to the specified file.

- **Prototype**

```
int ql_fwrite(void * buffer, size_t size, size_t num, QFILE fd)
```

- **Parameter**

buffer:
[In] The memory address for storing the data to be written, which cannot be a null pointer.

size:
[In] The size of each element to be written. Unit: byte.

num:
[In] The number of the elements to be written. The final size of data written is *size* × *num*.

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

The size of data written	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.6. ql_fseek

This function moves the file pointer to a specific position in a file.

- **Prototype**

```
int ql_fseek(QFILE fd, long offset, int origin)
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

offset:
[In] The offset of the file pointer. Unit: byte.

origin:
[In] The starting position to add the offset, including the following values:
QL_SEEK_SET The beginning of the file.
QL_SEEK_CUR The current position of the file pointer.
QL_SEEK_END The end of the file.

- **Return Value**

The relative byte offsets of the file pointer from the beginning of a file	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.7. ql_frewind

This function moves the file pointer to the beginning of a file.

- **Prototype**

```
int ql_frewind(QFILE fd)
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.8. ql_ftell

This function returns relative byte offsets of the file pointer from the beginning of a file.

- **Prototype**

```
int ql_ftell(QFILE fd);
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

The byte offsets of the file pointer	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* for details.

3.3.9. ql_fstat

This function gets the file status with the file handle.

- **Prototype**

```
int ql_fstat(QFILE fd, struct stat *st)
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

st:
[Out] The structure of the file status, which cannot be a null pointer. See *stat* structure in the C standard library for details.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.10. ql_stat

This function gets the file status with the file name.

- **Prototype**

```
int ql_stat(const char *file_name, struct stat *st)
```

- **Parameter**

file_name:
[In] The file name (with path), whose format is consistent with the one in *ql_fopen()*.

st:
[Out] The structure of the file status, which cannot be a null pointer. See *stat* structure in the C standard library for details.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.11. ql_ftruncate

This function truncates a file from the beginning of the file to the specified length.

- **Prototype**

```
int ql_ftruncate(QFILE fd, uint length)
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

length:
[In] The specified length for truncating the file. Unit: byte.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.12. ql_fsize

This function gets the file size. Unit: byte.

- **Prototype**

```
int ql_fsize(int fd)
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

File size	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* for details.

3.3.13. ql_file_exist

This function checks whether a file exists.

- **Prototype**

```
int ql_file_exist(const char *file_path)
```

- **Parameter**

file_path:
[In] The file name (with path), whose format is consistent with the one in *ql_fopen()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.14. ql_mkdir

This function creates a directory.

- **Prototype**

```
int ql_mkdir(const char *path, uint mode)
```

- **Parameter**

path:
[In] The directory name (with path), whose format is consistent with *file_name* in *ql_fopen()*.

mode:
[In] Invalid parameter. It is designed to be close to the C standard library. Enter 0 or a positive number.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.15. ql_opendir

This function opens a specified directory and returns the directory handle.

- **Prototype**

```
QDIR *ql_opendir(const char *path)
```

- **Parameter**

path:
[In] The directory name (with path), whose format is consistent with *file_name* in *ql_fopen()*.

- **Return Value**

Directory handle	Successful execution
NULL	Failed execution

3.3.16. ql_closedir

This function closes a specified directory.

- **Prototype**

```
int ql_closedir(QDIR *pdir)
```

- **Parameter**

pdir:
[In] The directory handle, that is, the return value after successful execution of *ql_opendir()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.17. ql_readdir

This function gets file information from the specified directory. See ***Chapter 3.3.17.1*** for *qdirent.*

- **Prototype**

```
qdirent * ql_readdir(QDIR *pdir)
```

- **Parameter**

pdir:
[In] The directory handle, that is, the return value after successful execution of *ql_opendir()*.

- **Return Value**

Non-null pointer to *qdirent*	Successful execution
NULL	Failed execution

3.3.17.1. qdirent

The structure of file information:

```
typedef struct
{
    int d_ino;
    uint8 d_type;
    char d_name[256];
}qdirent;
```

- **Parameter**

Type	Parameter	Description
int	*d_ino*	Index node number.
uint8	*d_type*	File type.
char	*d_name*	File name. The maximum length is 255 characters.

3.3.18. ql_telldir

This function returns the position of the specified directory stream.

- **Prototype**

```
int ql_telldir(QDIR *pdir);
```

- **Parameter**

pdir:
[In] The directory handle, that is, the return value after successful execution of *ql_opendir()*.

- **Return Value**

The directory stream position	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.19. ql_seekdir

This function sets the directory stream read position.

- **Prototype**

```
int ql_seekdir(QDIR *pdir, int offset)
```

- **Parameter**

pdir:
[In] The directory handle, that is, the return value after successful execution of *ql_opendir()*.

offset:
[In] The offset of the directory stream relative to the beginning of the file. Unit: byte.

- **Return Value**

Directory stream read location set	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.20. ql_rewinddir

This function moves the directory stream read position to the beginning of the directory.

- **Prototype**

```
int ql_rewinddir(QDIR *pdir)
```

- **Parameter**

pdir:
[In] The directory handle, that is, the return value after successful execution of *ql_opendir()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.21. ql_rmdir

This function erases the specified empty directory.

- **Prototype**

```
int ql_rmdir(const char *path)
```

- **Parameter**

path:
[In] The directory name (with path), whose format is consistent with *file_name* in *ql_fopen()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.22. ql_rename

This function renames a file.

- **Prototype**

```
int ql_rename(const char *oldname, const char *newname)
```

- **Parameter**

oldname:
[In] The original file name (with path), whose format is consistent with *file_name* in *ql_fopen()*.

newname:
[In] The new file name (with path), whose format is consistent with *file_name* in *ql_fopen()*.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.23. ql_fs_free_size

This function gets the size of the remaining space in the indicated partition with a file (with path), directory path, or partition name. Unit: byte.

- **Prototype**

```
int64 ql_fs_free_size(const char *path)
```

- **Parameter**

path:
[In] File (with path), directory path, or partition name, such as UFS:a.txt, UFS, or SD.

- **Return Value**

The size of the remaining space in the indicated partition	Successful execution
Other values	Failed execution

3.3.24. ql_nvm_fwrite

This function writes a simple configuration file.

- **Prototype**

```
int ql_nvm_fwrite(const char *config_file_name, void *buffer, size_t size, size_t num)
```

- **Parameter**

config_file_name:
[In] The simple configuration file name without path.

buffer:
[In] The memory address for storing the data to be written, which cannot be a null pointer.

size:
[In] The size of each element to be written. Unit: byte.

num:
[In] The number of the elements to be written. The final size of data written is *size* × *num*.

- **Return Value**

The size of data written	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.25. ql_nvm_fread

This function reads a simple configuration file.

- **Prototype**

```
int ql_nvm_fread(const char *config_file_name, void *buffer, size_t size, size_t num)
```

- **Parameter**

config_file_name:
[In] The simple configuration file name without path.

buffer:
[In] The memory address for storing the data to be read, which cannot be a null pointer.

size:
[In] The size of each element to be read. Unit: byte.

num:
[In] The number of the elements to be read. The final size of data read is *size* × *num*.

- **Return Value**

The size of data read	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.26. ql_sfs_setkey

This function configures the SFS file key for reading and writing.

- **Prototype**

```
int ql_sfs_setkey(void *key, uint8_t len)
```

- **Parameter**

key:
[In] The memory address where the key is stored. It cannot be a null pointer.

len:
[In] The length of the key to be set. Range: 1–16. Unit: byte.

- **Return Value**

See *ql_file_errcode_e* in the header file for details.

3.3.27. ql_fs_free_size_by_fd

This function gets the remaining space of the indicated partition with the file handle. Unit: byte.

- **Prototype**

```
int64 ql_fs_free_size_by_fd(int fd);
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

The remaining space of the indicated partition	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.28. ql_fputc

This function writes a character to a file.

- **Prototype**

```
int ql_fputc(int chr, int fd);
```

- **Parameter**

chr:
[In] The ASCII value of the character to be written.

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

The ASCII value of the written character	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.29. ql_fgetc

This function reads a character from a file.

- **Prototype**

```
int ql_fgetc(int fd);
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

The ASCII value of the read character	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.30. ql_ferror

This function checks whether the current file handle is valid.

- **Prototype**

```
int ql_ferror(int fd);
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

0	The current file handle is valid.
Other values	Invalid input. See *ql_file_errcode_e* in the header file for details.

3.3.31. ql_fsync

This function synchronizes the modified file data in memory to a storage device.

- **Prototype**

```
int ql_fsync(int fd);
```

- **Parameter**

fd:
[In] The file handle, that is, the return value after successful execution of *ql_fopen()*.

- **Return Value**

0	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.32. ql_cust_nvm_fwrite

This function writes a user-defined simple configuration file.

- **Prototype**

```
int ql_cust_nvm_fwrite(void *buffer, size_t size, size_t num);
```

- **Parameter**

buffer:
[In] The memory address for storing the data to be written. It cannot be a null pointer.

size:
[In] The size of each element to be written. Unit: byte.

num:
[In] The number of elements to be written. The final size of data written is *size* × *num*.

- **Return Value**

The size of data written	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

3.3.33. ql_cust_nvm_fread

This function reads a user-defined simple configuration file.

- **Prototype**

```
int ql_cust_nvm_fread(void *buffer, size_t size, size_t num);
```

- **Parameter**

buffer:
[In] The memory address for storing the data to be read. It cannot be a null pointer.

size:
[In] The size of each element to be read. Unit: byte.

num:
[In] The number of the elements to be read. The final size of data read is *size* × *num*.

- **Return Value**

The size of data read	Successful execution.
Other values	Failed execution. See *ql_file_errcode_e* in the header file for details.

4 Demo

ql_fs_demo.c, the file system API demo, is in the *components\ql-application\fs* directory. You can refer to the demo to implement related functions.

5 Appendix References

Table 3: Related Document

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen_CSDK_Quick_Start_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
EFS	External File System
RTOS	Real Time Operating System
SDK	Software Development Kit
SD	Secure Digital Memory Card/SD card
SFFS	Small-File-Oriented File System
SFS	Secure File System
SPI	Serial Peripheral Interface
UFS	User File System